

• `interview_prep.sql`

-- 21 patterns. one way of thinking.

SQL Pattern Recognition & LeetCode Interview Guide

Infosys / Software Developer Interview Preparation

A revision handbook built around pattern recognition — so that when a new SQL problem appears in the interview, you recognize the shape of it before you write a single line of code.

Table of Contents

Section 1	SQL Logical Execution Order
Section 2	The 21 SQL Patterns
Section 3	Complete Problem-to-Pattern Map
Section 4	Most Important Patterns for Interviews
Section 5	How to Recognize the Pattern — Decision Guide

Use this guide top to bottom the first time through, then jump straight to Section 5 during last-minute revision — it is the fastest way back into every pattern.

Section 1 — SQL Logical Execution Order

The order you *type* a query is not the order SQL actually *runs* it. Knowing the real order is what makes WHERE vs HAVING, and where a subquery is allowed to reference which column, make sense instantly.

SELECT clauses are written in this order:

FROM

↓

JOIN

↓

ON

↓

WHERE

↓

GROUP BY

↓

AGGREGATE

↓

HAVING

↓

SELECT

↓

DISTINCT

↓

ORDER BY

↓

LIMIT

#	Clause	What it actually does
1	FROM	Pick the starting table(s). Everything else operates on what FROM brings in.
2	JOIN	Attach rows from another table.
3	ON	Decide exactly how the two tables' rows are matched up.
4	WHERE	Throw away individual rows that don't meet a condition — before any grouping happens.
5	GROUP BY	Collapse the surviving rows into groups.
6	Aggregate functions	Compute one value per group: COUNT(), SUM(), AVG(), MIN(), MAX().
7	HAVING	Throw away entire groups that don't meet a condition — this runs after grouping/aggregating.
8	SELECT	Choose which columns/expressions actually appear in the output.
9	DISTINCT	Remove duplicate rows.

9	DISTINCT	Remove duplicate output rows.
10	ORDER BY	Sort the final rows.
11	LIMIT	Cut the result down to a fixed number of rows.

Important clarification: aggregate functions such as `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()` are *operations* you use inside a `SELECT` or `HAVING` clause — they are not a separate clause of their own. They simply need `GROUP BY` (or the whole table) to know which rows to work over.

Section 2 — The 21 SQL Patterns

Every LeetCode/interview SQL problem you will see is a variation on one of these 21 shapes. Learn to spot the shape, and the query almost writes itself.

PATTERN 1 Basic JOIN

TIER 1 - MUST KNOW

WHEN TO USE

Use a **JOIN** whenever the answer needs columns that live in two different, related tables. If the question mentions two entities (a person and their address, a product and its sale), you almost certainly need a JOIN.

TEMPLATE

```
Template

SELECT ...
FROM table1 t1
JOIN table2 t2
  ON t1.id = t2.id;
```

PROBLEMS FROM YOUR NOTES

- Combine Two Tables
- Product Sales Analysis I

EXAMPLE QUERY — COMBINE TWO TABLES

```
Combine Two Tables

SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a
  ON p.personId = a.personId;
```

HOW IT WORKS

- **p** and **a** are aliases — short nicknames for Person and Address so we don't repeat the full table name.
- A **LEFT JOIN** is used here (instead of a plain JOIN) so that every person still shows up even if they have no address row — the address columns simply come back as NULL.
- The **ON** clause tells SQL how the two tables are related: matching rows where **personId** is the same.

EASY PATTERN TO REMEMBER

```
Need data from another table
↓
JOIN
```

Product Sales Analysis I

```
Product Sales Analysis I

SELECT p.product_name, s.year, s.price
FROM Sales s
INNER JOIN Product p
  ON s.product_id = p.product_id;
```

PATTERN 2 LEFT JOIN + IS NULL

TIER 1 · MUST KNOW

WHEN TO USE

Use this pattern whenever the question asks for rows in Table A that have **no matching row** in Table B — for example "customers who never ordered" or "employees with no bonus".

TEMPLATE

```
SELECT ...  
FROM A  
LEFT JOIN B  
    ON A.id = B.id  
WHERE B.id IS NULL;
```

PROBLEMS FROM YOUR NOTES

- Customers Who Never Order
- Employee Bonus

EXAMPLE QUERY — CUSTOMERS WHO NEVER ORDER

```
SELECT c.name AS Customers  
FROM Customers c  
LEFT JOIN Orders o  
    ON c.id = o.customerId  
WHERE o.customerId IS NULL;
```

HOW IT WORKS

- Start with **all** customers (that's what LEFT JOIN guarantees — every row from the left table survives).
- For a customer with no matching order, every column that came from Orders — including customerId — comes back as **NULL**.
- Filtering with WHERE o.customerId IS NULL keeps only the customers whose order side was empty.

EASY PATTERN TO REMEMBER

```
LEFT JOIN + IS NULL  
↓  
"Who has NO matching record?"
```

Employee Bonus

```
SELECT e.name AS name, b.bonus AS bonus  
FROM Employee e  
LEFT JOIN Bonus b  
    ON e.empId = b.empId  
WHERE b.bonus < 1000  
    OR b.bonus IS NULL;
```

PATTERN 3 GROUP BY + COUNT + HAVING

TIER 1 · MUST KNOW

WHEN TO USE

This is one of the most tested SQL interview patterns. Use it whenever the question talks about groups of rows and puts a **condition on the group itself** — phrases like "at least 3", "more than 5", "duplicate", or "number of orders".

TEMPLATE

```
SELECT column
FROM table
GROUP BY column
HAVING COUNT(*) >= condition;
```

PROBLEMS FROM YOUR NOTES

- Duplicate Emails
- Actors and Directors Who Cooperated At Least Three Times
- Managers with at Least 5 Direct Reports

EXAMPLE QUERY — DUPLICATE EMAILS

```
SELECT email AS Email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

HOW IT WORKS

- GROUP BY email collapses every row that shares the same email into a single group.
- COUNT(*) counts how many original rows fell into each group.
- HAVING COUNT(*) > 1 keeps only groups with more than one row — i.e. duplicates. This filter runs **after** grouping, which is exactly why we use HAVING and not WHERE (see the comparison table in the Key Distinctions box).

EASY PATTERN TO REMEMBER

```
Same email
↓
GROUP BY email
↓
COUNT
↓
more than 1 → duplicate
```

Actors and Directors Who Cooperated At Least Three Times

```
SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING COUNT(*) >= 3;
```

Here we group by **two** columns at once because it's the *combination* (actor, director) that must repeat three times — not either column alone.

Managers with at Least 5 Direct Reports

Managers with 5+ Reports

```
SELECT e.name
FROM Employee e
WHERE e.id IN (
    SELECT e2.managerId
    FROM Employee e2
    GROUP BY e2.managerId
    HAVING COUNT(*) >= 5
);
```

The inner query groups employees by `managerId` to count direct reports, then the outer query looks up the manager's own name by matching their `id` against that list.

PATTERN 4 GROUP BY + Aggregate Function

TIER 1 - MUST KNOW

WHEN TO USE

Use this whenever you need one summary value **per group** — the earliest date per player, the total sales per region, the average score per class.

PROBLEMS FROM YOUR NOTES

- Game Play Analysis I

EXAMPLE QUERY — GAME PLAY ANALYSIS I

Game Play Analysis I

```
SELECT player_id,
       MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

HOW IT WORKS

- `GROUP BY player_id` creates one bucket of rows per player.
- `MIN(event_date)` then finds the smallest (earliest) date inside each bucket — that player's first login.

Important: `MIN()`, `MAX()`, `AVG()`, `SUM()`, and `COUNT()` are *aggregate functions* — operations, not a separate SQL clause. `GROUP BY` is the clause; the aggregate function is what you compute inside each group.

EASY PATTERN TO REMEMBER

```
GROUP BY person
    ↓
MIN → smallest / earliest
MAX → largest / latest
AVG → average
SUM → total
COUNT → number
```

PATTERN 5 GROUP BY + COUNT + ORDER BY + LIMIT

TIER 1 - MUST KNOW

WHEN TO USE

Use this when the question asks for the single group with the **highest or lowest** count — "the customer with the most orders", "the product that sold the least".

PROBLEMS FROM YOUR NOTES

- Customer Placing the Largest Number of Orders

EXAMPLE QUERY — CUSTOMER PLACING THE LARGEST NUMBER OF ORDERS

🔴 🟡 🟢 Largest Number of Orders

```
SELECT customer_number
FROM Orders
GROUP BY customer_number
ORDER BY COUNT(*) DESC
LIMIT 1;
```

HOW IT WORKS

- Group all orders by customer.
- Count how many orders fall into each customer's group.
- Sort the groups from highest count to lowest with `ORDER BY COUNT(*) DESC`.
- `LIMIT 1` keeps only the top row — the customer with the most orders.

EASY PATTERN TO REMEMBER

```
"Largest number of orders"
  ↓
GROUP BY customer
  ↓
COUNT(*)
  ↓
ORDER BY COUNT(*) DESC
  ↓
LIMIT 1
```

PATTERN 6 MAX / MIN + Subquery

TIER 1 · MUST KNOW

WHEN TO USE

Use this whenever you need to find the highest or lowest value and then **compare against it** — classic examples are "second highest salary" or "the largest value below X".

PROBLEMS FROM YOUR NOTES

- Second Highest Salary

EXAMPLE QUERY — SECOND HIGHEST SALARY — SOLUTION 1 (ORDER BY + OFFSET)

🔴 🟡 🟢 Second Highest Salary (v1)

```
SELECT (
  SELECT DISTINCT salary
  FROM Employee
  ORDER BY salary DESC
  LIMIT 1 OFFSET 1
) AS SecondHighestSalary;
```

HOW IT WORKS

- `DISTINCT` removes duplicate salary values so tied salaries don't take up two "ranks".
- Sorting `DESC` puts the highest salary first, so `OFFSET 1` skips it and lands on the second-highest.
- Wrapping it in an outer `SELECT ( ... )` means that if there is no second-distinct salary, the whole thing simply returns `NULL` instead of erroring.

EASY PATTERN TO REMEMBER

```
Find MAX salary
↓
Find salaries below MAX
↓
Find MAX of those
↓
Second highest
```

Second Highest Salary — Solution 2 (MAX + Subquery)

```
Second Highest Salary (v2)

SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (
    SELECT MAX(salary)
    FROM Employee
);
```

This version follows the "MAX + Subquery" idea directly: find the overall MAX, keep only salaries strictly below it, then take the MAX of what's left. This same idea extends naturally to "third highest" (nest one level deeper) or "maximum salary below \$X".

PATTERN 7 LIMIT + OFFSET (Nth Highest)

TIER 2 · VERY USEFUL

WHEN TO USE

Use this when the question generalizes "second highest" into an arbitrary **Nth highest** value, usually inside a reusable function.

PROBLEMS FROM YOUR NOTES

- Nth Highest Salary

EXAMPLE QUERY — NTH HIGHEST SALARY

```
Nth Highest Salary

CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
    SET N = N - 1;

    RETURN (
        SELECT DISTINCT salary
        FROM Employee
        ORDER BY salary DESC
        LIMIT 1 OFFSET N
    );
END
```

HOW IT WORKS

- SQL's OFFSET is zero-indexed, but "Nth highest" is one-indexed (the 1st highest is the very top row).
- So the function first converts N into an offset by subtracting 1, then reuses the exact same ORDER BY + LIMIT 1 OFFSET pattern from Pattern 6.

EASY PATTERN TO REMEMBER

1st → OFFSET 0
2nd → OFFSET 1
3rd → OFFSET 2
4th → OFFSET 3

Formula: OFFSET = N - 1

PATTERN 8 Window Function / Ranking

TIER 2 · VERY USEFUL

WHEN TO USE

Use this whenever the question explicitly asks for a **rank** next to each row, rather than just sorting the rows.

PROBLEMS FROM YOUR NOTES

- Rank Scores

EXAMPLE QUERY — RANK SCORES

```
Rank Scores

SELECT score,
       DENSE_RANK() OVER (ORDER BY score DESC) AS `rank`
FROM Scores;
```

HOW IT WORKS

- OVER (ORDER BY score DESC) tells SQL to rank every row by score, highest first, without collapsing rows the way GROUP BY does.
- DENSE_RANK() assigns the rank, but importantly it does **not** leave gaps after a tie.

DENSE_RANK vs RANK: for scores 100, 90, 90, 80 — DENSE_RANK gives 1, 2, 2, 3 (no gap after the tie), while plain RANK gives 1, 2, 2, 4 (the tie "uses up" rank 3). Most "rank scores" style questions want DENSE_RANK.

EASY PATTERN TO REMEMBER

Need ranking
↓
DENSE_RANK()

PATTERN 9 Self Join

TIER 1 · MUST KNOW

WHEN TO USE

Use a **self join** whenever you need to compare two rows from the *same* table — for example an employee row against that same employee's manager row, which also lives in the Employee table.

TEMPLATE

```
Template

FROM Employee e
JOIN Employee m
  ON e.managerId = m.id
```

PROBLEMS FROM YOUR NOTES

- Employees Earning More Than Their Managers

EXAMPLE QUERY — EMPLOYEES EARNING MORE THAN THEIR MANAGERS

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m
    ON e.managerId = m.id
WHERE e.salary > m.salary;
```

HOW IT WORKS

- The same Employee table is opened twice under two different aliases: **e** stands for the employee row, **m** stands for that employee's manager row.
- The join condition `e.managerId = m.id` lines each employee up with their manager.
- The WHERE clause then compares the two salaries directly.

EASY PATTERN TO REMEMBER

```
Same table
+
different rows
+
comparison
↓
SELF JOIN
```

PATTERN 10 Self Join + Consecutive Rows

TIER 3 - SPECIFIC

WHEN TO USE

Use this when you need to look at a row together with the row (or rows) that immediately follow it in the same table — e.g. "find 3 rows in a row with the same value".

PROBLEMS FROM YOUR NOTES

- Consecutive Numbers

EXAMPLE QUERY — CONSECUTIVE NUMBERS

```
SELECT DISTINCT a.num AS ConsecutiveNums
FROM Logs a
JOIN Logs b
    ON b.id = a.id + 1
JOIN Logs c
    ON c.id = b.id + 1
WHERE a.num = b.num
AND b.num = c.num;
```

HOW IT WORKS

- The same table is joined to itself three times: **a** is the current row, **b** is the next row (`id + 1`), **c** is the row after that (`id + 2`).
- The WHERE clause then checks that all three rows hold the same number.
- DISTINCT avoids listing the same qualifying number more than once if it appears in several overlapping runs.

EASY PATTERN TO REMEMBER

```
a → current  
b → next  
c → next-next
```

```
a.id  
  ↓ +1  
b.id  
  ↓ +1  
c.id
```

Then: a.num = b.num = c.num

PATTERN 11 Self Join + Date Comparison

TIER 3 · SPECIFIC

WHEN TO USE

Use this when a row needs to be compared against the row for the **previous (or next) date** in the same table.

PROBLEMS FROM YOUR NOTES

- Rising Temperature

EXAMPLE QUERY — RISING TEMPERATURE

```
SELECT w.id  
FROM Weather w  
JOIN Weather w2  
  ON w2.recordDate = DATE_SUB(w.recordDate, INTERVAL 1 DAY)  
WHERE w.temperature > w2.temperature;
```

HOW IT WORKS

- **w** represents "today's" row, **w2** represents "yesterday's" row.
- `DATE_SUB(w.recordDate, INTERVAL 1 DAY)` computes what yesterday's date would be for the current row, and the `JOIN` finds the row that actually has that date.
- The `WHERE` clause then checks that today's temperature is higher than yesterday's.

EASY PATTERN TO REMEMBER

```
Same table  
+  
previous/next date  
  ↓  
SELF JOIN
```

PATTERN 12 Correlated Subquery

TIER 2 · VERY USEFUL

WHEN TO USE

Use this when the inner query's answer **depends on the current row** of the outer query — you'll typically see the inner query referencing a column from the outer table, such as `e2.departmentId = e.departmentId`.

PROBLEMS FROM YOUR NOTES

- Department Highest Salary

EXAMPLE QUERY — DEPARTMENT HIGHEST SALARY

Department Highest Salary

```
SELECT d.name AS Department,
       e.name AS Employee,
       e.salary AS Salary
FROM Employee e
JOIN Department d
  ON e.departmentId = d.id
WHERE e.salary = (
  SELECT MAX(e2.salary)
  FROM Employee e2
  WHERE e2.departmentId = e.departmentId
);
```

HOW IT WORKS

- For every candidate employee **e** in the outer query, the inner query re-runs, restricted to **e2** rows that share the same department.
- It then finds the MAX salary within just that department.
- The outer WHERE keeps only employees whose salary equals their own department's maximum.

WHERE vs correlated subquery: a plain WHERE compares a row to a constant or to the result of an independent subquery. A *correlated* subquery re-evaluates once per outer row because it references that outer row's columns — it cannot be run on its own.

EASY PATTERN TO REMEMBER

```
For current employee e
  ↓
look at same department
  ↓
find MAX salary
  ↓
compare with e.salary
```

PATTERN 13 Correlated COUNT() / Top N

TIER 2 · VERY USEFUL

WHEN TO USE

Use this to find the **top N** rows per group without an explicit ranking function — e.g. "top 3 salaries per department".

PROBLEMS FROM YOUR NOTES

- Department Top Three Salaries

EXAMPLE QUERY — DEPARTMENT TOP THREE SALARIES

Department Top Three Salaries

```
SELECT d.name AS Department,
       e.name AS Employee,
       e.salary AS Salary
FROM Employee e
JOIN Department d
  ON e.departmentId = d.id
WHERE (
  SELECT COUNT(DISTINCT e2.salary)
  FROM Employee e2
  WHERE e2.departmentId = e.departmentId
  AND e2.salary > e.salary
) < 3;
```

HOW IT WORKS

- For the current employee **e**, the inner query counts how many *distinct* salaries in the same department are strictly higher than theirs.
- If fewer than 3 salaries beat theirs, then this employee must be sitting in 1st, 2nd, or 3rd place — i.e. inside the Top 3.

Why DISTINCT matters here: if two employees tie for the highest salary, we don't want that tie to be counted twice and accidentally push a real 3rd-place salary out of the Top 3.

EASY PATTERN TO REMEMBER

```
COUNT(DISTINCT higher values) < N
      ↓
    Top N
```

PATTERN 14 DELETE + Self Join

TIER 2 · VERY USEFUL

WHEN TO USE

Use this when you must **delete** duplicate rows but keep exactly one copy of each — typically the one with the smallest id.

PROBLEMS FROM YOUR NOTES

- Delete Duplicate Emails

EXAMPLE QUERY — DELETE DUPLICATE EMAILS

```
DELETE p
FROM Person p
JOIN Person d
  ON p.email = d.email
 AND p.id > d.id;
```

HOW IT WORKS

- p** represents the row we intend to delete; **d** represents another row that happens to share the same email.
- The condition `p.id > d.id` makes sure we only ever delete the row with the *larger* id, so the row with the smallest id per email always survives.
- `DELETE p` tells MySQL which of the two joined aliases to actually remove.

EASY PATTERN TO REMEMBER

```
Same value
+
larger ID
  ↓
DELETE duplicate
```

PATTERN 15 NOT IN + Subquery

TIER 2 · VERY USEFUL

WHEN TO USE

Use this whenever the question asks you to **exclude** a group of ids found by another query — "salespeople who never sold to company X".

PROBLEMS FROM YOUR NOTES

- Sales Person

EXAMPLE QUERY — SALES PERSON

```
SELECT s.name
FROM SalesPerson s
WHERE s.sales_id NOT IN (
    SELECT o.sales_id
    FROM Orders o
    JOIN Company c
      ON c.com_id = o.com_id
   WHERE c.name = 'RED'
);
```

HOW IT WORKS

- The inner query first finds every `sales_id` connected to an order placed with the company named 'RED'.
- The outer query then keeps only salespeople whose id is **not** in that list.

Caution with NOT IN: if the subquery can return a NULL value, `NOT IN` can unexpectedly return zero rows. When NULLs are possible, prefer `NOT EXISTS` instead. In this problem `sales_id` is never NULL, so `NOT IN` is safe.

EASY PATTERN TO REMEMBER

```
Find salespersons who sold to RED
      ↓
NOT IN
      ↓
Salespersons who never sold to RED
```

PATTERN 16 CASE / IF

TIER 1 - MUST KNOW

WHEN TO USE

Use CASE or IF whenever the output depends on a **condition** — the question uses words like "if", "otherwise", "classify", "yes/no", or "swap".

PROBLEMS FROM YOUR NOTES

- Triangle Judgement
- Exchange Seats
- Swap Sex of Employees

EXAMPLE QUERY — TRIANGLE JUDGEMENT

```
SELECT *,
       IF(
           x + y > z
          AND x + z > y
          AND y + z > x,
          'Yes',
          'No'
       ) AS triangle
FROM Triangle;
```

HOW IT WORKS

- `IF(condition, value_if_true, value_if_false)` is a compact way to write a single yes/no check.
- Three sides form a triangle only if each side is shorter than the sum of the other two — all three checks must hold at once, hence `AND`.

EASY PATTERN TO REMEMBER

```
IF / CASE
  ↓
condition
  ↓
different result
```

Use when the question says: if, otherwise, when, swap, classify, yes/no

Swap Sex of Employees

```
UPDATE Salary
SET sex = CASE
  WHEN sex = 'm' THEN 'f'
  ELSE 'm'
END;
```

CASE is used here inside an UPDATE: every 'm' becomes 'f', and everything else (i.e. 'f') falls into ELSE and becomes 'm'.

Exchange Seats (preview — full explanation in Pattern 19)

```
SELECT
  CASE
    WHEN id % 2 = 1
      AND id < (SELECT COUNT(*) FROM Seat)
      THEN id + 1
    WHEN id % 2 = 0
      THEN id - 1
    ELSE id
  END AS id,
  student
FROM Seat
ORDER BY id;
```

PATTERN 17 GROUP BY + COUNT(*) = 1

TIER 3 · SPECIFIC

WHEN TO USE

Use this when you need to find values that appear **exactly once** in a table, then pick the largest (or smallest) of those.

PROBLEMS FROM YOUR NOTES

- Biggest Single Number

EXAMPLE QUERY — BIGGEST SINGLE NUMBER

```
SELECT (
  SELECT num
  FROM MyNumbers
  GROUP BY num
  HAVING COUNT(*) = 1
  ORDER BY num DESC
  LIMIT 1
) AS num;
```

HOW IT WORKS

- GROUP BY num collapses repeated values together; HAVING COUNT(*) = 1 keeps only numbers that were never repeated.
- Sorting those survivors DESC and taking LIMIT 1 gives the largest one.
- Wrapping everything in an outer SELECT (...) is what makes this safe: if no number is ever unique, the inner query returns zero rows, and the scalar outer SELECT quietly returns **NULL** instead of throwing an error.

EASY PATTERN TO REMEMBER

```
Need largest unique value
↓
GROUP BY
↓
HAVING COUNT(*) = 1
↓
ORDER BY DESC
↓
LIMIT 1
```

PATTERN 18 Modulo / Odd-Even

TIER 3 · SPECIFIC

WHEN TO USE

Use the modulo operator % whenever a question distinguishes rows by an **odd or even** id or position.

PROBLEMS FROM YOUR NOTES

- Not Boring Movies

EXAMPLE QUERY — NOT BORING MOVIES

```
SELECT *
FROM Cinema
WHERE id % 2 = 1
      AND description != 'boring'
ORDER BY rating DESC;
```

HOW IT WORKS

- id % 2 gives the remainder after dividing the id by 2 — 1 means odd, 0 means even.
- Combined with the description filter, this keeps only odd-id rows that are not boring, then sorts by rating.

EASY PATTERN TO REMEMBER

```
id % 2 = 1 → odd
id % 2 = 0 → even

Very common interview condition.
```

PATTERN 19 Swap Consecutive IDs

TIER 3 · SPECIFIC

WHEN TO USE

This is a more specific relative of Pattern 16 (CASE), worth knowing on its own: use it when adjacent rows (1↔2, 3↔4, ...) need to trade places.

PROBLEMS FROM YOUR NOTES

- Exchange Seats

EXAMPLE QUERY — EXCHANGE SEATS

```
Exchange Seats

SELECT
  CASE
    WHEN id % 2 = 1
      AND id < (SELECT COUNT(*) FROM Seat)
      THEN id + 1
    WHEN id % 2 = 0
      THEN id - 1
    ELSE id
  END AS id,
  student
FROM Seat
ORDER BY id;
```

HOW IT WORKS

- An odd id that is *not* the last row moves up by one (it takes the next seat).
- An even id always moves down by one (it takes the previous seat) — this is the natural partner of the rule above.
- An odd id that *is* the last row has nobody to swap with, so it stays the same — that's what the ELSE id branch protects against.

EASY PATTERN TO REMEMBER

```
ODD + NOT LAST → +1
EVEN           → -1
LAST ODD       → SAME
```

Example:

```
1 → 2
2 → 1
3 → 4
4 → 3
5 → 5
```

PATTERN 20 COUNT(DISTINCT) = Total Count

TIER 3 · SPECIFIC

WHEN TO USE

Use this whenever the question asks for entities that satisfy **every single item** in another list — "customers who bought all products", "users who completed every required task".

PROBLEMS FROM YOUR NOTES

- Customers Who Bought All Products

EXAMPLE QUERY — CUSTOMERS WHO BOUGHT ALL PRODUCTS

```
SELECT customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (
    SELECT COUNT(*)
    FROM Product
);
```

HOW IT WORKS

- Group each customer's purchase rows together, then count how many *different* products they bought with `COUNT(DISTINCT product_key)`.
- Compare that against the total number of products that exist at all, from the subquery `SELECT COUNT(*) FROM Product`.
- If a customer's distinct-product count equals the grand total, they must have bought one of everything.

Worked example: suppose the Product table has 3 products in total. Customer 1 bought products 1, 2, and 3 → 3 distinct products = 3 total → qualifies. Customer 2 only bought products 1 and 2 → 2 distinct products ≠ 3 total → does not qualify.

EASY PATTERN TO REMEMBER

```
Bought ALL
↓
COUNT(DISTINCT bought)
=
COUNT(total)
```

PATTERN 21 COUNT / COUNT + ROUND (Fraction)

TIER 3 · SPECIFIC

WHEN TO USE

Use this whenever the answer is a **fraction or rate** — "what percentage of users came back", "what fraction of players logged in again".

PROBLEMS FROM YOUR NOTES

- Game Play Analysis IV

EXAMPLE QUERY — GAME PLAY ANALYSIS IV

```
SELECT ROUND(
    COUNT(*) /
    (SELECT COUNT(DISTINCT player_id) FROM Activity),
    2
) AS fraction
FROM Activity a
WHERE a.event_date = (
    SELECT DATE_ADD(MIN(a2.event_date), INTERVAL 1 DAY)
    FROM Activity a2
    WHERE a.player_id = a2.player_id
);
```

HOW IT WORKS

- For each player, the correlated subquery finds their very first login date, then adds one day to get "the day after they first logged in".
- The outer WHERE keeps only activity rows that actually happened on that exact next day — meaning that player

did return.

- `COUNT(*)` counts how many players returned; dividing by the total distinct players gives the fraction, and `ROUND(..., 2)` keeps it to two decimal places.

EASY PATTERN TO REMEMBER

```
First login
↓
First login + 1 day
↓
Did player return?
↓
COUNT returned players
↓
÷ total players
↓
ROUND(..., 2)

fraction = successful / total
```

Section 3 — Complete Problem-to-Pattern Map

A single scannable table covering every problem in your notes — perfect for a 10-minute refresh right before the interview.

LeetCode Problem	Main Pattern	Important SQL
Combine Two Tables	Basic JOIN	LEFT JOIN ... ON
Product Sales Analysis I	Basic JOIN	INNER JOIN ... ON
Customers Who Never Order	LEFT JOIN + IS NULL	LEFT JOIN ... WHERE ... IS NULL
Employee Bonus	LEFT JOIN + IS NULL	LEFT JOIN + IS NULL OR <
Duplicate Emails	GROUP BY + COUNT + HAVING	GROUP BY ... HAVING COUNT(*) > 1
Actors and Directors Who Cooperated At Least Three Times	GROUP BY + COUNT + HAVING	GROUP BY two columns
Managers with at Least 5 Direct Reports	GROUP BY + COUNT + HAVING	Subquery + GROUP BY + HAVING
Game Play Analysis I	GROUP BY + Aggregate	GROUP BY ... MIN()
Customer Placing the Largest Number of Orders	GROUP BY + ORDER BY + LIMIT	ORDER BY COUNT(*) DESC LIMIT 1
Second Highest Salary	MAX/MIN + Subquery	LIMIT 1 OFFSET 1 / MAX + WHERE <
Nth Highest Salary	LIMIT + OFFSET	OFFSET = N - 1
Rank Scores	Window Function	DENSE_RANK() OVER (ORDER BY ...)
Employees Earning More Than Their Managers	Self Join	JOIN Employee m ON managerId = id
Consecutive Numbers	Self Join + Next Row	JOIN ... ON id + 1
Rising Temperature	Self Join + Date	DATE_SUB(..., INTERVAL 1 DAY)
Department Highest Salary	Correlated Subquery	WHERE salary = (correlated MAX)
Department Top Three Salaries	Correlated COUNT / Top N	COUNT(DISTINCT higher) < 3
Delete Duplicate Emails	DELETE + Self Join	DELETE p ... ON email AND id >
Sales Person	NOT IN	sales_id NOT IN (subquery)
Triangle Judgement	CASE / IF	IF(condition, 'Yes', 'No')
Swap Sex of Employees	CASE / IF	UPDATE ... SET sex = CASE
Biggest Single Number	GROUP BY + COUNT(*) = 1	HAVING COUNT(*) = 1, ORDER, LIMIT
Not Boring Movies	Modulo / Odd-Even	id % 2 = 1
Exchange Seats	Swap Consecutive IDs	CASE WHEN id % 2 ...
Customers Who Bought All Products	COUNT(DISTINCT) = Total	HAVING COUNT(DISTINCT) = (SELECT COUNT(*))
Game Play Analysis IV	COUNT / COUNT + ROUND	ROUND(COUNT(*)/COUNT(DISTINCT), 2)

Section 4 — Most Important Patterns for Interviews

Don't try to memorize all 21 patterns with equal intensity. Prioritize like this:

TIER 1 — Must Know

These eight ideas cover the large majority of interview SQL questions. If you only have one evening to revise, revise these.

- JOIN
- LEFT JOIN + IS NULL
- GROUP BY + COUNT + HAVING
- GROUP BY + Aggregate
- GROUP BY + ORDER BY + LIMIT
- Subquery (MAX/MIN)
- Self Join
- CASE / IF

TIER 2 — Very Useful

Come up often enough that you should recognize them instantly, but they build directly on Tier 1 concepts.

- Correlated Subquery
- COUNT(DISTINCT)
- NOT IN
- DENSE_RANK
- LIMIT + OFFSET
- DELETE + Self Join

TIER 3 — Specific Patterns

Narrower, situation-specific patterns. Good to have seen once so nothing looks unfamiliar on interview day.

- Consecutive rows
- Date comparison
- Exchange IDs
- Fraction calculation
- All products condition

Reminder: every Tier 2 and Tier 3 pattern above is explained in full, with a template, a worked example, and a memory diagram, back in Section 2 — use this section as an index of *priority*, not as a replacement for the full explanations.

Section 5 — How to Recognize the Pattern

This is the core purpose of this entire document. When you see a brand-new SQL problem, resist the urge to jump straight to writing a query. First ask what the question is really asking for, using this checklist:

Need data from another table?

→ `JOIN`

Need records with NO matching record?

→ `LEFT JOIN + IS NULL`

Need a count per person/group?

→ `GROUP BY + COUNT`

Need a condition on the group itself?

→ `HAVING`

Need the highest/lowest value?

→ `MAX / MIN / ORDER BY`

Need to compare two rows from the same table?

→ `SELF JOIN`

Need the current row compared with related rows?

→ `CORRELATED SUBQUERY`

Need a ranking?

→ `DENSE_RANK / RANK`

Need conditional output?

→ `CASE / IF`

Need to exclude something?

→ `NOT IN / NOT EXISTS`

Need ALL items to be present?

→ `COUNT(DISTINCT) = total COUNT`

Key Distinctions Worth Memorizing

WHERE vs HAVING	WHERE filters individual rows before grouping. HAVING filters entire groups after grouping/aggregating.
JOIN vs SELF JOIN	A JOIN combines two different tables. A SELF JOIN combines a table with itself, under two aliases, to compare rows within it.
GROUP BY vs ORDER BY	GROUP BY collapses rows into groups for aggregation. ORDER BY only changes the display order of the final rows — it never merges rows.
RANK vs DENSE_RANK	Both rank rows by a value. RANK leaves a gap after a tie (1, 2, 2, 4); DENSE_RANK does not (1, 2, 2, 3).
WHERE vs correlated subquery	A plain WHERE compares against a constant or an independent subquery. A correlated subquery re-runs once per outer row because it references that row's own columns.

COUNT(*) vs COUNT(DISTINCT column)

COUNT(*) counts every row, including duplicates. COUNT(DISTINCT column) counts only the unique values in that column.

IN vs NOT IN

IN keeps rows whose value appears in a list/subquery. NOT IN keeps rows whose value does not — but beware of NULLs in the subquery, which can make NOT IN return zero rows unexpectedly.

Once you've identified the pattern, jump back to its full write-up in Section 2 — template, worked example, and memory diagram are all there, ready to adapt to the exact wording of whatever problem is in front of you. Good luck with the Infosys interview.